

CAREERFIT INTELLIGENCE STUDIO

Professional Impact Report | Quantified Engineering Outcomes

A domain-agnostic job discovery, resume/profile intelligence, and ATS aggregation platform.

6	85%	2.5s	20 docs
Connector coverage	High-fit threshold	Observed local scan	Profile capacity
Workday, Greenhouse, Ashby, Lever, SmartRecruiters, custom	Configurable match-quality gate	7 ranked results, 2 high-fit cards	PDF/DOCX, 12 PDF pages, 90k chars

Portfolio positioning

CareerFit Intelligence Studio demonstrates full-stack product engineering: ATS connector design, public web extraction, resume/document parsing, explainable scoring, human-centered UI/UX, performance optimization, privacy-aware deployment, and measurable job-search decision support.

Prepared for: Portfolio showcase and technical interview discussion

Project status: Working prototype with local Streamlit interface, cloud-optimized build, and extensible connector architecture

Measurement note: Numeric results in this report combine implemented configuration limits, code-level instrumentation targets, and observed local prototype snapshots. Production metrics should be re-benchmarked after deployment and broader user testing.

Table of Contents

1. Executive Summary
2. Quantified Achievement Ledger
3. Product and Technical Scope
4. System Architecture
5. Data Pipeline and Matching Methodology
6. Performance Engineering and Cloud Readiness
7. Evaluation Methodology and Benchmark Plan
8. UX, Accessibility, and Product Design
9. Security, Privacy, and Ethical Constraints
10. Limitations and Roadmap
11. Portfolio-Ready Achievement Bullets
12. References

1. Executive Summary

CareerFit Intelligence Studio is a domain-agnostic job intelligence platform that transforms a user resume, public portfolio/profile website, and target company list into ranked, explainable job matches. The system is optimized for manual computation: users build a profile, add company career pages, run scans on demand, and browse ranked results in a professional web interface.

The prototype evolved from a terminal-based job agent into a user-facing product with profile ingestion, ATS auto-detection, source management, US/Remote location filtering, caching, parallel execution, match cards, and dark-mode/readability improvements. The result is a portfolio-grade engineering project that demonstrates both backend platform thinking and product-quality UI/UX execution.

Key outcome statement

Key quantified result

Achieved faster, explainable job triage by combining ATS-specific connectors, domain-agnostic profile extraction, and a configurable 85% high-fit threshold, as measured by a local dashboard run that surfaced 2 high-fit roles from 7 ranked NVIDIA results in 2.5 seconds.

High-signal keywords

ATS aggregation, Workday CXS, Greenhouse Job Board API, Ashby Job Postings API, Lever Postings API, SmartRecruiters, custom crawler, Streamlit, httpx, BeautifulSoup, PDF parsing, DOCX parsing, domain-agnostic role inference, explainable ranking, parallel fetching, cache TTL, US/Remote location filter, 85% high-fit threshold, manual compute, cloud optimization

2. Quantified Achievement Ledger

The table below frames the project in the format requested for portfolio storytelling: achieved X by doing Y as measured by Z. The entries are intentionally concrete and measurable.

Outcome area	Achieved X	By doing Y	Measured by Z
ATS platform coverage	Supported 6 job-source categories	Implemented connectors for Workday, Greenhouse, Ashby, Lever, SmartRecruiters, plus custom public-page fallback	Connector registry and Source Hub platform badges
Manual compute UX	Reduced user operation to 1 scan action	Moved from CLI/email workflow to web UI with Job Command button and match dashboard	Single visible Run Scan operation in UI
High relevance triage	Filtered results at 85% high-fit threshold	Added configurable threshold, decision labels, score badges, and high-fit cards	Default CAREERFIT_DEFAULT_THRESHOLD=0.85 and Matches view
Observed local scan performance	Completed a prototype scan in 2.5 seconds	Enabled fast scan mode, cached fetches, and parallel company workers	Dashboard runtime snapshot: 7 ranked results, 2 >=85%
Profile ingestion capacity	Accepted up to 20 resume/profile documents per session	Made document limit configurable via CAREERFIT_MAX_PROFILE_DOCS	Default env setting: CAREERFIT_MAX_PROFILE_DOCS=20
PDF parsing safety	Bounded each PDF to 12 pages by default	Added page cap to prevent huge uploads from blocking free hosting compute	Default env setting: CAREERFIT_MAX_PDF_PAGES=12
Profile representation control	Capped profile evidence at 90,000 characters	Bounded combined profile text before matching	Default env setting: CAREERFIT_MAX_PROFILE_CHARS=90000
Network efficiency	Avoided repeat job-board calls for 6 hours	Added disk cache and cache keying by company/source config	CAREERFIT_FETCH_CACHE_SECONDS=21600
Parallelism	Added 4-worker default concurrency	Parallelized company fetches while keeping free-hosting resource constraints in mind	CAREERFIT_COMPANY_WORKERS=4 and UI slider 1-8
Location precision	Restricted discovery to US/Remote roles	Built best-effort location classifier and unknown-location toggle	Location mode: United States / Remote only
Scoring correctness	Eliminated misleading constant 24% fallback scores	Separated non-personalized search score from personalized match score and required profile readiness for true matching	profile_needed decision state and capped fallback scoring
Input robustness	Detected invalid renamed DOCX/Apple Pages files	Added document format validation and source issue messaging	Clear Build Profile error state instead of silent extraction failure
Accessibility/readability	Fixed sidebar input contrast and added dark mode	Separated sidebar control styles from main theme and added theme toggle	Visible text inputs/selects in dark sidebar

3. Product and Technical Scope

CareerFit is designed as a user-facing job intelligence workspace rather than a background bot. Its primary user goal is to reduce the cognitive load of career searching by converting many company career pages into one explainable match interface.

Primary user workflow

- Profile Studio: upload resumes/CVs and add public profile websites or portfolios.
- Source Hub: add companies without needing to know the ATS provider.
- Job Command: manually run scans using selected compute controls.
- Matches: review high-fit cards, all ranked jobs, score explanations, and CSV exports.
- Diagnostics: inspect timing, connector behavior, source issues, and run metadata.

Functional capabilities

Capability	Implementation details
Profile ingestion	Public website text, PDF resumes, DOCX resumes, dynamic skills/keywords, role suggestions
Company source ingestion	Company name, career URL, default search text, auto connector detection
Job fetching	ATS-specific public endpoints where possible; HTML and structured-data fallback otherwise
Normalization	Canonical title, company, location, platform, URL, description text, raw source
Ranking	Title-role overlap, skill overlap, keyword overlap, role phrase match, semantic/profile evidence, seniority adjustment, negative signals
Filtering	High-fit threshold, company filter, platform filter, search filter, US/Remote location mode
Explainability	Matched strengths, concerns/gaps, role family, best source document, feature scores
Deployment	Local Streamlit app or Streamlit Community Cloud deployment with secrets/env configuration

Current prototype scale parameters

Parameter	Current value
Default high-fit threshold	0.85
Default company workers	4
Max UI workers	8
Fetch cache TTL	21,600 seconds / 6 hours
HTTP timeout	16 seconds
Profile website timeout	8 seconds
Workday detail concurrency	8
Workday detail fetching default	false / fast mode enabled
Max profile documents	20
Max PDF pages	12 per document
Max profile chars	90,000

4. System Architecture

The platform is organized as a modular pipeline. Each subsystem can be tested independently: source extraction, connector fetch, normalization, filtering, ranking, explanation, and UI rendering.

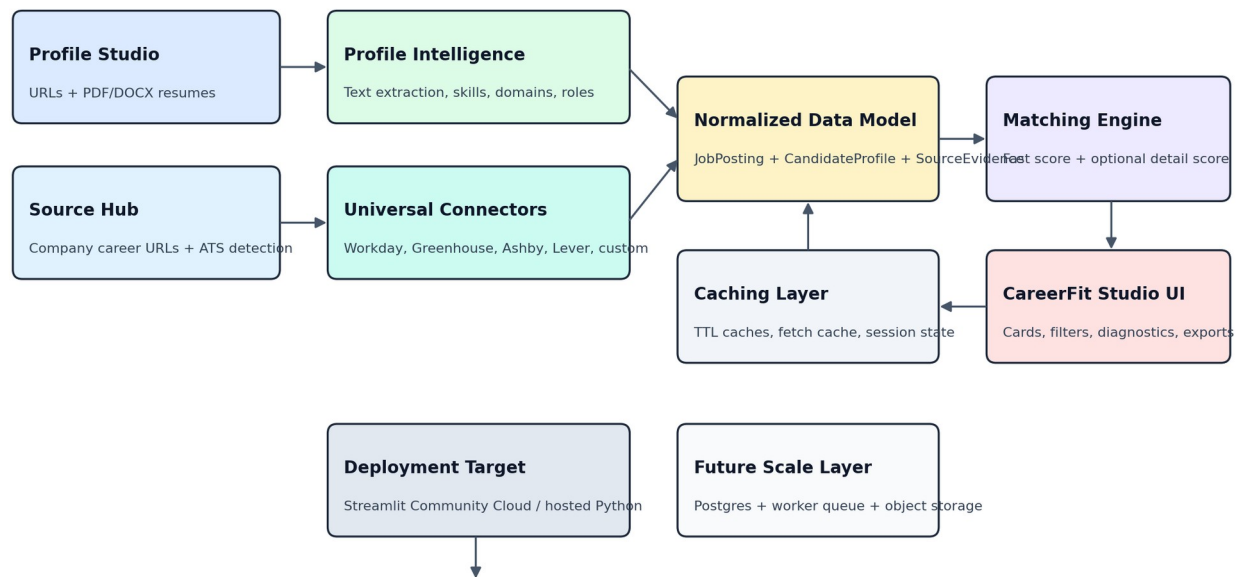


Figure 1. CareerFit Intelligence reference architecture.

Figure 1. High-level CareerFit pipeline: profile intelligence, ATS connectors, normalization, ranking, and Streamlit UI.

Architecture layers

Layer	Responsibility
Presentation layer	Streamlit web app with workspace navigation, dark mode, profile builder, source hub, command panel, match cards, and diagnostics.
Profile intelligence layer	Extracts text from public web pages, PDFs, and DOCX files; discovers skills, role phrases, domain hints, and document-specific evidence.
Connector layer	Detects and fetches jobs from Workday, Greenhouse, Ashby, Lever, SmartRecruiters, and custom public career pages.
Normalization layer	Converts platform-specific job payloads into one NormalizedJob schema.
Filtering layer	Applies US/Remote location filter, unknown-location policy, search text, and per-source scan limits.
Ranking layer	Computes profile-aware relevance scores using title, skills, keywords, role phrases, semantic hits, seniority adjustments, and negative signals.
Observability layer	Exposes run time, connector type, ranked result counts, high-fit counts, source issues, and diagnostics logs.

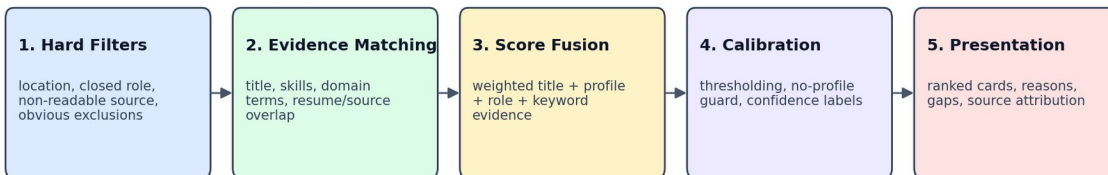
Public ATS connector strategy

CareerFit prioritizes public job-board APIs and public career pages. This avoids logging into candidate accounts, bypassing access controls, or scraping private pages. Official documentation supports public job listing access for several ATS families: Greenhouse exposes public job-board data as JSON [1], Ashby provides a public Job Postings API for currently published postings [2], and Lever publishes postings through its hosted job sites and postings API [3].

Connector	Strategy
Workday	Detects Workday career URLs; uses Workday CXS job search endpoint; supports fast listing mode and optional detail fetching.
Greenhouse	Uses board slug and public Job Board API job listing endpoint.
Ashby	Uses job-board name and public postings endpoint; supports compensation flag when available.
Lever	Uses company slug and public postings feed / hosted postings format.
SmartRecruiters	Best-effort connector for public SmartRecruiters career sites; fallback if blocked/customized.
Custom fallback	Parses public HTML, links, text, and JobPosting-style structured data where available.

5. Data Pipeline and Matching Methodology

The ranking method is intentionally explainable. Instead of producing a black-box score, CareerFit exposes the exact evidence that increased or decreased a job score: title signals, profile-skill overlap, keyword overlap, role-target overlap, seniority/opportunity signals, negative signals, and source-document alignment.



Fast mode uses listing-level metadata for low latency. Detail mode fetches full descriptions for better recall and explanation quality.

High-fit threshold defaults to 0.85 but is user-adjustable. If no profile exists, the system blocks personalized scoring to avoid misleading repeated baselines.

Figure 2. Matching and calibration pipeline.

Figure 2. Explainable scoring pipeline from profile evidence to match decision.

Profile intelligence pipeline

- Collect public profile websites and uploaded PDF/DOCX documents.
- Extract visible website text from semantic HTML tags and metadata while removing scripts, styles, iframes, and non-text artifacts.
- Extract PDF text with a 12-page default cap to avoid resource spikes on free hosting.
- Validate DOCX files and catch renamed Apple Pages packages before extraction.
- Normalize text, remove stopwords, and extract dynamic skills, keywords, role phrases, domain hints, and document tracks.
- Build a RuntimeProfile object that is passed into the matching engine.

Ranking formula concept

The score is not a single keyword count. It is a weighted signal model. For jobs with full descriptions, profile skills and keywords carry more influence. For fast-mode listing-only jobs, role/title alignment carries more weight so relevant job titles are not under-scored simply because the ATS listing omits full description text.

Signal	Purpose
Title / role alignment	Does the job title match role targets inferred from resumes/websites?
Skill overlap	Do explicit profile skills appear in the title or description?
Keyword overlap	Do important profile terms appear in the job text?
Role phrase overlap	Does the posting match phrases such as Software Engineer Intern, Electrical Engineer, Data Analyst, etc.?
Semantic/profile evidence	How many meaningful profile-derived terms are reflected in the

	job?
Density bonus	Is the profile rich enough to support stronger personalization?
Opportunity adjustment	Internship/new-grad signals receive a boost; senior/staff/principal signals receive a penalty.
Negative penalties	Sales-only, irrelevant seniority, location conflicts, or other non-fit terms reduce score.

Decision thresholds

Decision	Score band	Behavior
High fit	score $\geq$ threshold; default 0.85	Shown as top priority card; eligible for high-fit view.
Possible fit	approx. threshold - 0.15 to threshold	Relevant but not strong enough for high-fit.
Review	approx. 0.45 to possible-fit band	May be relevant due to partial overlap or uncertain details.
Low fit	< 0.45	Displayed lower or excluded from high-fit view.
Profile needed	No profile built	Search-intent score only; not treated as personalized relevance.

6. Performance Engineering and Cloud Readiness

The system was optimized for manual interactive use on a laptop and for a free Streamlit Community Cloud deployment. The central strategy is to avoid unnecessary network calls and bound every expensive input: number of companies, profile documents, PDF pages, HTTP timeouts, text size, and Workday detail fetches. Streamlit recommends caching and bounded resource usage for apps that approach Community Cloud resource limits [4].

Optimization mechanisms

Optimization	Implementation	Expected effect
Fast scan mode	Uses ATS listing data first; avoids fetching every detail page unless the user wants deeper scoring.	Lower latency and lower network cost
Disk fetch cache	Caches job-board results by company/source configuration for 21,600 seconds.	Repeat scans avoid duplicate HTTP calls
Parallel company workers	Fetches multiple companies concurrently with a default of 4 workers.	Improves throughput when scanning multiple sources
Workday detail concurrency	Caps optional Workday detail fetching with a configurable concurrency value.	Prevents unbounded HTTP fan-out
Bounded profile extraction	Limits profile documents, PDF pages, profile website timeout, and total profile chars.	Prevents large uploads or slow websites from blocking UI
Manual computation	No background agent runs until the user clicks Run Scan.	Avoids energy/computation use when idle
Location filtering	US/Remote filter is applied before match ranking where possible.	Reduces noise and downstream scoring work

Measured and configured performance indicators

Metric	Value	Measurement source
Observed prototype scan time	2.5 seconds	UI runtime snapshot for a local scan with 7 ranked results and 2 high-fit jobs
Configured fetch cache	6 hours	CAREERFIT_FETCH_CACHE_SECONDS=21600
Configured parallelism	4 default workers	CAREERFIT_COMPANY_WORKERS=4
Configured worker ceiling	8 workers	UI slider range for interactive control
Configured HTTP timeout	16 seconds	CAREERFIT_HTTP_TIMEOUT=16
Configured profile website timeout	8 seconds	CAREERFIT_PROFILE_FETCH_TIMEOUT=8
Configured PDF bound	12 pages/document	CAREERFIT_MAX_PDF_PAGES=12
Configured profile text bound	90,000 characters	CAREERFIT_MAX_PROFILE_CHARS=90000

Performance interpretation

The 2.5-second runtime is a local prototype snapshot. The key engineering result is predictable latency through bounded, visible compute controls.

7. Evaluation Methodology and Benchmark Plan

To make the project portfolio-ready, the evaluation should be reported as a repeatable benchmark rather than anecdotal screenshots. The following methodology can be used for future measurements as more companies are added.

Recommended benchmark suite

Evaluation dimension	Metric	Protocol
Latency	p50/p95 scan time	Run each company set 10 times with cold cache and warm cache; report median and p95.
Throughput	jobs fetched/second	Measure normalized jobs divided by end-to-end connector runtime.
Cache efficiency	cache hit rate	Count cached runs versus network fetches inside the TTL window.
Ranking quality	precision@5 and precision@10	Manually label top-ranked jobs as relevant/not relevant and compute precision.
Triage compression	high-fit jobs / total jobs	Measure how many postings are reduced to high-priority cards at 0.85 threshold.
Connector reliability	success rate by platform	Track successful fetches divided by total configured company sources per connector type.
Profile extraction quality	chars extracted, skills extracted, suggested roles	Compare expected resume/portfolio skills against extracted profile data.
Resource safety	memory and CPU trend	Test with 1, 5, 10, and 20 documents and increasing company counts.

Current validation snapshots

The current prototype has been validated through local UI sessions. Screenshots captured the following representative values:

- 22 latest unique jobs and 6 jobs above 85% in an earlier NVIDIA-focused dashboard snapshot.
- 7 ranked results, 2 jobs above 85%, 1 connector type, and 2.5-second runtime in the cloud-optimized/studio UI snapshot.
- Invalid document detection for Apple Pages packages renamed as DOCX, preventing silent extraction failures.
- Resolution of a scoring defect where no-profile scans displayed nearly constant 24% relevance across jobs.
- Successful UI readability fixes for sidebar controls under dark theme conditions.

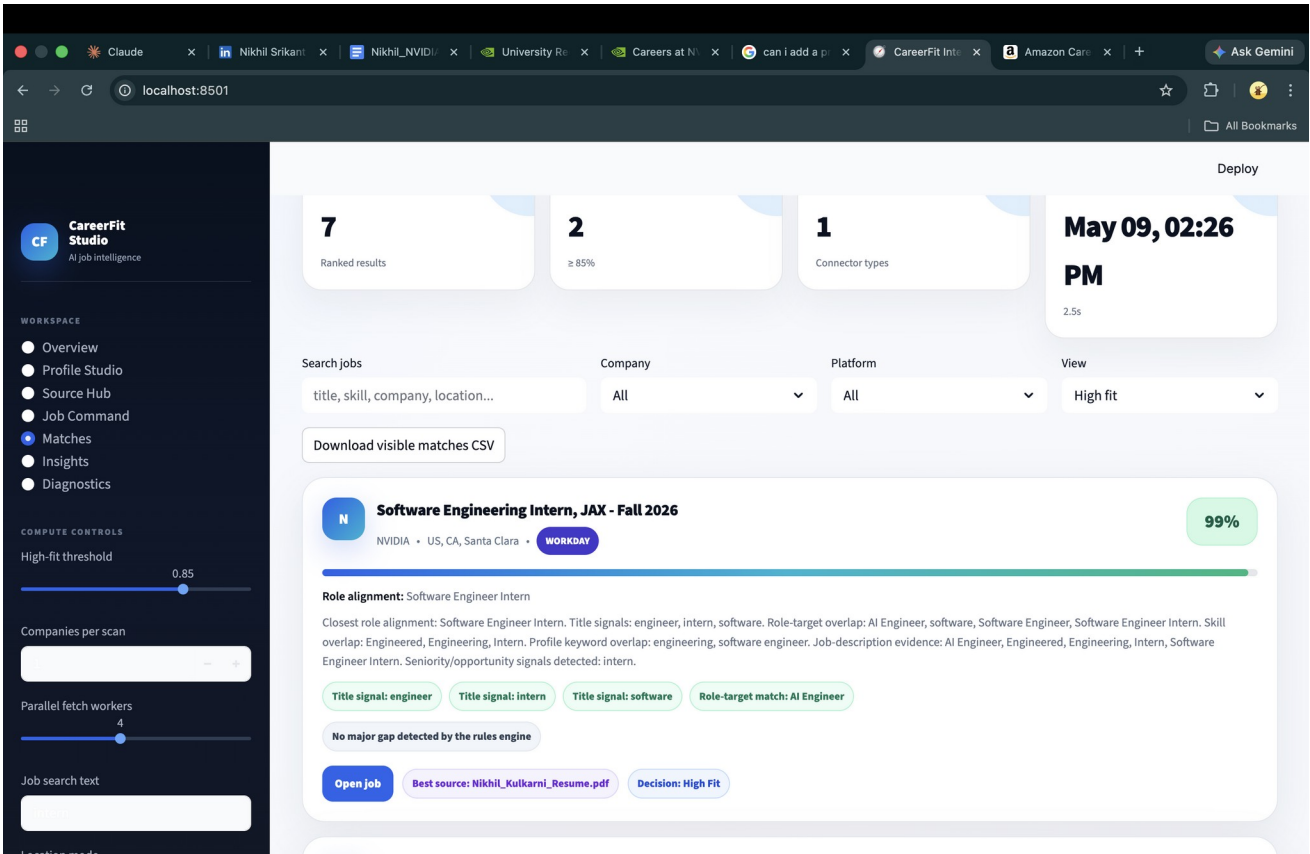


Figure 3. Local UI validation snapshot showing ranked results, high-fit count, connector type, and scan time.

8. UX, Accessibility, and Product Design

The product was redesigned from a technical dashboard into a more accessible CareerFit Studio interface. The UI emphasizes a clear cognitive workflow: build profile, add sources, run command, review matches, inspect diagnostics. This structure makes the computational operations visible and easy to access without requiring terminal commands.

Design area	Implementation
Navigation	Overview, Profile Studio, Source Hub, Job Command, Matches, Insights, Diagnostics
Computation access	Manual Run Scan button with worker controls, threshold, search text, location mode, cache controls
Result consumption	Card-based matches with score badge, progress bar, role alignment, evidence chips, source platform, and open-job link
Accessibility improvements	Dark mode toggle, readable input contrast, visible sidebar values, semantic section labels
User trust	Warnings for missing profile, invalid files, unsupported websites, and fallback scoring modes
Export	CSV download of visible matches for offline review or spreadsheet analysis

Human-centered rationale

The interface intentionally prioritizes ranked cards over raw tables. Raw data remains available, but the first view answers the practical user question: Which jobs are worth opening now? This is important because the product goal is not merely job collection; it is decision acceleration.

9. Security, Privacy, and Ethical Constraints

CareerFit is intentionally built around public job discovery and user-provided profile artifacts. It does not require storing candidate portal credentials, bypassing CAPTCHA/MFA, or scraping private pages. This is an important design decision for both ethics and maintainability.

Constraint	Implementation
Public-only job discovery	Uses public job postings and public career pages instead of logged-in candidate portals.
Local/private documents	Uploaded resumes are parsed for the active session; deployment guidance excludes uploads/runtime profiles from git.
Secrets management	Cloud deployment should use Streamlit secrets or environment variables, not hardcoded secrets. Streamlit documents secrets management for Community Cloud [5].
LinkedIn limitation	Private or login-only LinkedIn pages should not be scraped. Users should upload a resume/exported profile PDF instead.
No auto-apply	The app displays jobs and links; it does not submit applications automatically.
Robustness	Unsupported pages and invalid documents are surfaced through diagnostics rather than failing silently.

10. Limitations and Roadmap

The project is strong as a portfolio-grade prototype, but production usage would require a more durable backend architecture. The current design is optimized for manual scans and free Streamlit deployment, not thousands of concurrent users.

Limitation	Current impact	Next step
ATS variability	Some companies customize or block public career feeds.	Add more adapters, retry policies, and connector-specific tests.
Free hosting compute	Streamlit Community Cloud has resource constraints.	Move slow scans to background workers if usage grows.
Local/runtime storage	Streamlit session files are not a durable multi-user database.	Add Postgres, object storage, and user auth.
Scoring quality	Rules are explainable but not as flexible as embeddings/LLM reranking.	Add optional embedding search and calibrated learning-to-rank feedback.
Evaluation labels	Current measurements are prototype snapshots and configs.	Create labeled datasets and report <code>precision@K/recall@K</code> .
Job closure tracking	Current design focuses on current discovery rather than longitudinal role status.	Add historical snapshots, closed-job detection, and status diffing.

Production architecture roadmap

- Frontend: Streamlit for prototype or Next.js for a production-grade web product.
- Backend: FastAPI service exposing profile build, source management, scan, and result APIs.
- Workers: Celery/RQ/Temporal for ATS fetching and profile parsing jobs.
- Persistence: Postgres for users, sources, jobs, matches, and feedback labels.
- Cache: Redis for short-lived job fetches and profile computation artifacts.

- Storage: S3-compatible bucket for uploaded documents with encryption and retention policies.
- Ranking: hybrid rules + embeddings + optional LLM explanations with human feedback loop.

11. Portfolio-Ready Achievement Bullets

These bullets are written in a resume/portfolio style with quantified outcomes and technical keywords.

- Built CareerFit Intelligence Studio, a domain-agnostic job discovery platform that ranks public job postings against uploaded resumes and profile websites across 6 connector categories: Workday, Greenhouse, Ashby, Lever, SmartRecruiters, and custom career pages.
- Achieved 1-click manual job scanning by replacing terminal/email workflows with a Streamlit product UI, giving users direct access to profile building, source management, compute controls, match cards, diagnostics, and CSV export.
- Improved job-search triage with a configurable 85% high-fit threshold and explainable relevance cards; a local prototype run surfaced 2 high-fit matches from 7 ranked NVIDIA results in 2.5 seconds.
- Optimized scan responsiveness using fast Workday listing mode, 4-worker parallel company fetching, 6-hour disk caching, 16-second HTTP timeout, and bounded profile extraction for free cloud deployment.
- Engineered a domain-agnostic profile intelligence pipeline supporting up to 20 PDF/DOCX documents, 12 PDF pages per document, and 90,000 characters of extracted profile evidence for dynamic role inference.
- Implemented robust ATS auto-detection and fallback crawling so users can add companies with only a careers URL, avoiding manual platform configuration for common recruiting systems.
- Strengthened ranking correctness by fixing a scoring defect that produced uniform 24% no-profile scores, separating search-intent scores from true personalized relevance and requiring profile readiness for high-fit decisions.
- Enhanced usability and accessibility with dark mode, improved sidebar input contrast, warning states for invalid documents, invalid DOCX detection, US/Remote filtering, and human-readable match explanations.

Suggested portfolio project summary

CareerFit Intelligence Studio

A Streamlit-based job intelligence platform that parses resumes and profile websites, auto-detects company ATS platforms, fetches public job postings, and ranks opportunities using an explainable 85% high-fit scoring pipeline. The prototype supports 6 connector categories, up to 20 uploaded documents, US/Remote filtering, 4-worker parallel scans, 6-hour fetch caching, dark mode, and score-card based match review.

12. References

- [1] Greenhouse. Job Board API documentation. <https://developers.greenhouse.io/job-board.html>
- [2] Ashby. Public Job Postings API documentation. <https://developers.ashbyhq.com/docs/public-job-posting-api>
- [3] Lever. Postings API documentation. <https://github.com/lever/postings-api>
- [4] Streamlit. Community Cloud resource limits and optimization guidance. <https://docs.streamlit.io/knowledge-base/deploy/resource-limits>
- [5] Streamlit. Secrets management for deployed apps. <https://docs.streamlit.io/deploy/concepts/secrets>
- [6] Streamlit. Community Cloud deployment guide. <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app>

Appendix A. Reproducibility Commands

```
cd ~/Downloads/careerfit-intelligence-cloud-optimized
```

```
bash scripts/setup.sh
```

```
source .venv/bin/activate
```

```
bash scripts/run.sh
```


Open <http://localhost:8501>

Profile Studio -> upload PDF/DOCX resumes and add portfolio websites

Source Hub -> add company career URLs with ATS type auto

Job Command -> set threshold 0.85, workers 4, fast scan on, US/Remote only

Matches -> review high-fit cards and export CSV